Отчёт по заданию курса «Суперкомпьютерное моделирование и технологии»

Вариант 16

Васильченко Дмитрий Дмитриевич, группа 609

Ноябрь 2026

Содержание

1 Математическая постановка задачи **1**

2 Численный метод **2**
2.1 Метод фиктивных областей 2
2.2 Разностная схема 2
2.3 Вычисление S_{ij} 2
2.4 Метод сопряжённых градиентов 3

3 Последовательная программа **3**

4 OpenMP-программа **3**

5 MPI-программа **4**

6 Гибридная программа MPI+OpenMP **5**

7 Проверка корректности **5**

8 Запуск на ПВС IBM Polus **6**

9 Выводы **7**

1 Математическая постановка задачи

Требуется найти функцию $u(x, y)$, удовлетворяющую в области D уравнению Пуассона и граничному условию Дирихле на её границе γ:

$$-\Delta u = f(x, y), \quad (x, y) \in D, \qquad u(x, y) = \varphi(x, y), \quad (x, y) \in \gamma. \tag{1}$$

В варианте 16

$$D = \big\{(x, y) : x^2 + y^2 < 2\big\} \cap \big\{(x, y) : |y| < 1\big\}, \qquad f(x, y) = 1, \qquad \varphi(x, y) = \sin(x + y).$$

Область D — это часть круга радиуса $\sqrt{2}$ с центром в начале координат, заключённая в горизонтальной полосе $|y| < 1$. Граница γ состоит из двух горизонтальных отрезков $y = \pm 1$, $|x| \le 1$, и двух дуг окружности $x^2 + y^2 = 2$, $|y| \le 1$. В четырёх угловых точках $(\pm 1, \pm 1)$, где отрезки смыкаются с дугами, граница перестаёт быть гладкой. Область лежит в прямоугольнике $\Pi = [-\sqrt{2}, \sqrt{2}] \times [-1, 1]$.

2 Численный метод

2.1 Метод фиктивных областей

Область D погружается в прямоугольник $\Pi = [-\sqrt{2}, \sqrt{2}] \times [-1, 1]$, и вместо (1) в Π решается задача

$$-\Delta v + Q(x,y)\big(v - v_0\big) = F(x,y), \qquad v = v_0 \text{ на границе } \Pi,$$

где $Q = 0$, $F = f$ в D и $Q = 1/\varepsilon^2$, $F = 0$ в фиктивной области $\Pi \setminus D$. В качестве продолжения граничной функции взято $v_0(x,y) = \sin(x+y)$ во всём Π. Решение v приближает u в D с точностью $O(\varepsilon)$, принято $\varepsilon = h^2$, $h = \max(h_1, h_2)$.

2.2 Разностная схема

На равномерной сетке

$$x_i = -\sqrt{2} + i\,h_1, \quad y_j = -1 + j\,h_2, \qquad h_1 = \frac{2\sqrt{2}}{M}, \quad h_2 = \frac{2}{N},$$

во внутренних узлах записывается пятиточечная схема

$$-\frac{w_{i+1,j} - 2w_{ij} + w_{i-1,j}}{h_1^2} - \frac{w_{i,j+1} - 2w_{ij} + w_{i,j-1}}{h_2^2} + Q_{ij} w_{ij} = F_{ij} + Q_{ij}\, v_0(x_i, y_j), \qquad (2)$$

$$Q_{ij} = \frac{1}{\varepsilon^2}\Big(1 - \frac{S_{ij}}{h_1 h_2}\Big), \qquad F_{ij} = \frac{S_{ij}}{h_1 h_2}\, f,$$

где S_{ij} — площадь части ячейки $[x_i - \frac{h_1}{2}, x_i + \frac{h_1}{2}] \times [y_j - \frac{h_2}{2}, y_j + \frac{h_2}{2}]$, лежащей в D. В узлах, где ячейка целиком внутри D, имеем $S_{ij} = h_1 h_2$, то есть $Q_{ij} = 0$, $F_{ij} = f$, а в целиком фиктивных узлах $S_{ij} = 0$, то есть $Q_{ij} = 1/\varepsilon^2$, $F_{ij} = 0$. Значения на границе Π известны ($w = v_0$) и переносятся в правую часть. Получается система $Aw = B$ с самосопряжённым положительно определённым оператором A.

2.3 Вычисление S_{ij}

Поскольку область D выпукла, если все четыре угла ячейки лежат в D, то ячейка целиком принадлежит D и $S_{ij} = h_1 h_2$. В противном случае площадь вычисляется численным интегрированием длины вертикального сечения по x:

$$S_{ij} = \int_{x_i - h_1/2}^{x_i + h_1/2} \ell(x)\, dx, \qquad \ell(x) = \Big|\, [-g(x), g(x)] \cap [y_j - \tfrac{h_2}{2}, y_j + \tfrac{h_2}{2}] \,\Big|,$$

где $g(x) = \min\bigl(1, \sqrt{2 - x^2}\,\bigr)$ — верхняя граница области по y при фиксированном x (минимум из полосы $|y| < 1$ и окружности $x^2 + y^2 = 2$). Интеграл по x берётся составной формулой средних прямоугольников с $K = 128$ подынтервалами, а длина сечения $\ell(x)$ по y вычисляется точно. Такой подход точен по y (где граница — прямая или пересечение с интервалом) и корректно сглаживает криволинейную границу по x.

2.4 Метод сопряжённых градиентов

Система $Aw = B$ решается методом сопряжённых градиентов с диагональным (Якоби) предобуславливателем

$$D_{ij} = \frac{2}{h_1^2} + \frac{2}{h_2^2} + Q_{ij},$$

начиная с $w^{(0)} = 0$ во внутренних узлах. Все скалярные произведения вычисляются во взвешенной энергетической норме $(u, v)_E = h_1 h_2 \sum_{i,j} u_{ij} v_{ij}$, $\|u\|_E = \sqrt{(u, u)_E}$. На каждой итерации

$$z = D^{-1} r, \qquad \alpha = \frac{(z, r)_E}{(Ap, p)_E}, \qquad w \mathrel{+}= \alpha p, \qquad r \mathrel{-}= \alpha A p, \qquad \beta = \frac{(z_{\text{new}}, r_{\text{new}})_E}{(z, r)_E}, \qquad p = z + \beta p.$$

Итерации прекращаются, когда $\|w^{(k+1)} - w^{(k)}\|_E = |\alpha|\,\|p\|_E < \delta$, $\delta = 10^{-6}$.

3 Последовательная программа

Программа написана на языке C. Последовательная и OpenMP-версии собираются из одного исходного файла (без флага OpenMP получается последовательная версия). Программа состоит из двух этапов: вычисление коэффициентов Q_{ij}, F_{ij} и правой части B_{ij} для всех внутренних узлов и решение системы методом сопряжённых градиентов. Сеточные функции хранятся в одномерных массивах размера $(M+1)(N+1)$, в граничных узлах хранятся значения v_0, поэтому разностный оператор A применяется ко всем внутренним узлам по единой формуле (2).

С диагональным предобуславливателем на сетках 20×20, 40×40 и 80×80 метод сошёлся за 53, 102 и 200 итераций, значение решения в точке $(0, 0)$ составило 0.37634, 0.37457 и 0.37478 соответственно. Контроль корректности программы приведён в разделе 7.

4 OpenMP-программа

Директивой `#pragma omp parallel for` распараллелены все циклы по внутренним узлам сетки: вычисление коэффициентов S_{ij}, Q_{ij}, F_{ij}, B_{ij} и все циклы метода сопряжённых градиентов. Скалярные произведения вычисляются с `reduction(+:...)` по скалярным накопителям.

На ПВС IBM Polus (процессоры POWER8, 20 ядер на узел, SMT8) программа компилировалась командой `xlc_r -O3 -qsmp=omp`. Для замеров ускорения OpenMP-версии каждой нити выделялось отдельное физическое ядро (задачи ставились в планировщик

LSF с запросом по одному слоту-ядру на нить). Результаты приведены в таблице 1. Время решения T_p — полное время работы программы на p нитях, ускорение $S(p) = T_1/T_p$, где T_1 — время последовательной версии.

Таблица 1: Результаты расчётов на ПВС IBM Polus (OpenMP-код).

Количество OpenMP-нитей	Число точек сетки ($M \times N$)	Число итераций	Время решения, с	Ускорение
1	400×600	1243	1.776	1.00
2	400×600	1243	0.861	2.07
4	400×600	1243	0.804	2.23
8	400×600	1243	0.414	4.34
16	400×600	1243	0.545	3.24
1	800×1200	2211	12.990	1.00
4	800×1200	2211	3.325	3.90
8	800×1200	2211	2.598	5.00
16	800×1200	2211	2.937	4.42
32	800×1200	2211	2.411	5.39

OpenMP-версия ограничена одним процессором (общей памятью узла): на больших числах нитей рост ускорения замедляется из-за пропускной способности памяти и эффектов NUMA, достигая $\approx 5.4\times$ на 32 нитях.

5 MPI-программа

В MPI-версии внутренние узлы сетки разбиваются на $P_x \times P_y = P$ прямоугольных доменов. Декартова топология создаётся вызовом `MPI_Cart_create`. Разбиение $P_x \times P_y$ выбирается так, чтобы домены были максимально близки к квадратным (минимизируется $|\log(\text{отношения сторон домена})|$), а размеры доменов отличаются не более чем на один узел. Каждый процесс хранит свой блок с теневыми гранями шириной в один узел. На каждой итерации перед применением оператора выполняется обмен гранями с четырьмя соседями через `MPI_Sendrecv`, столбцы пересылаются производным типом `MPI_Type_vector`. Глобальные скалярные произведения собираются через `MPI_Allreduce` (за одну итерацию — три свёртки, две из которых объединены в один вызов).

Результаты приведены в таблице 2, в скобках указана выбранная решётка процессов $P_x \times P_y$. Ускорение считается относительно последовательной версии.

Приведено наименьшее из трёх прогонов время. Ускорение MPI на отдельных конфигурациях (например 400×600 при 4 процессах или 800×1200 при 16) немонотонно: планировщик LSF может размещать процессы одной задачи на разных вычислительных узлах, и тогда обмен гранями идёт по сети InfiniBand, а не через общую память,

Таблица 2: Результаты расчётов на ПВС IBM Polus (MPI-код).

Количество MPI-процессов	Число точек сетки ($M \times N$)	Число итераций	Время решения, с	Ускорение
2 (1×2)	400×600	1243	0.851	2.09
4 (2×2)	400×600	1243	1.912	0.93
8 (2×4)	400×600	1243	0.370	4.81
16 (4×4)	400×600	1243	0.270	6.59
4 (2×2)	800×1200	2211	4.000	3.25
8 (2×4)	800×1200	2211	3.480	3.73
16 (4×4)	800×1200	2211	4.150	3.13
32 (4×8)	800×1200	2211	1.380	9.41

что заметно увеличивает накладные расходы. На 32 процессах достигнуто ускорение $\approx 9.4\times$.

6 Гибридная программа MPI+OpenMP

Гибридная версия собирается из того же исходного файла, что и MPI-версия, но с флагом `-qsmp=omp`. Среда инициализируется вызовом `MPI_Init_thread` с уровнем `MPI_THREAD_FUNNELED`: все обращения к MPI выполняются вне параллельных областей, а вычислительные циклы внутри каждого процесса распараллелены OpenMP. Таким образом, MPI отвечает за обмен между процессами (в том числе между узлами), а OpenMP — за распараллеливание внутри узла.

Результаты для фиксированного числа процессов и разного числа нитей на процесс приведены в таблице 3. Ускорение считается относительно последовательной версии, общее число задействованных ядер равно $P \cdot t$.

При фиксированном числе процессов добавление нитей устойчиво ускоряет счёт. На 32 ядрах (4 процесса $\times 8$ нитей) гибрид даёт $\approx 9.3\times$ — на уровне чистого MPI на 32 процессах, но с меньшим числом обменов между процессами.

7 Проверка корректности

Корректность проверялась двумя способами.

Совпадение версий. На каждой сетке последовательная, OpenMP, MPI и гибридная версии выдают одинаковое число итераций и совпадающие до последнего знака контрольные величины — интеграл решения $\sum_{ij} w_{ij}\, h_1 h_2$ и максимум $\max_{ij} w_{ij}$ (таблица 4). Это подтверждает, что распараллеливание не меняет вычисляемого решения.

Задача с известным решением. При компиляции с макросом `-DCHECK_EXACT` решается тестовая задача с точным решением $u = x^2 + y^2$ (для него $-\Delta u = -4$, то есть

Таблица 3: Результаты расчётов на ПВС IBM Polus (гибридный код MPI+OpenMP).

MPI-процессов	Нитей на процесс	Число точек сетки ($M \times N$)	Число итераций	Время решения, с	Ускорение
2	1	400×600	1243	0.933	1.90
2	2	400×600	1243	0.668	2.66
2	4	400×600	1243	0.499	3.56
2	8	400×600	1243	0.449	3.96
4	1	800×1200	2211	3.766	3.45
4	2	800×1200	2211	2.349	5.53
4	4	800×1200	2211	2.451	5.30
4	8	800×1200	2211	1.400	9.28

Таблица 4: Контрольные величины (идентичны для всех четырёх версий).

Сетка ($M \times N$)	Число итераций	$\sum_{ij} w_{ij} h_1 h_2$	$\max_{ij} w_{ij}$
80×80	200	0.9526919582	0.9986943813
400×600	1243	0.9584739321	0.9999998891
800×1200	2211	0.9595052258	0.9999999927

$f = -4$, $\varphi = x^2 + y^2$). Максимальное отклонение численного решения от точного во внутренних узлах области D составило $\mathrm{err}_C = 6.25 \cdot 10^{-6}$, что подтверждает правильность разностной схемы и решателя.

8 Запуск на ПВС IBM Polus

При работе с кластером выявлен ряд технических особенностей, которые стоит учитывать.

- Компиляция: `xlc_r -O3 -qsmp=omp` (последовательная/OpenMP), для MPI и гибрида среда Spectrum MPI подключается через `source /polusfs/setenv/setup.SMPI`, компилятор — `mpixlc`.
- Штатная утилита `mpisubmit.pl` не допускает 16 и 32 нити при одном процессе и упаковывает нити в SMT одного ядра, что непригодно для замеров ускорения OpenMP. Для корректных замеров задачи ставились напрямую через `bsub` с выделением отдельного ядра на каждую нить.
- Для гибридной версии необходимо явно задавать `OMP_PROC_BIND=false`: иначе OpenMP-рантайм привязывает нити всех MPI-процессов узла к одним и тем же ядрам, и счёт замедляется более чем в 100 раз.

9 Выводы

Задача варианта 16 решена методом фиктивных областей с пятиточечной разностной схемой и методом сопряжённых градиентов с диагональным предобуславливателем. Реализованы и исследованы четыре версии программы: последовательная, OpenMP, MPI и гибридная MPI+OpenMP. Корректность подтверждена совпадением результатов всех версий и тестом с известным точным решением ($\mathrm{err}_C = 6.25 \cdot 10^{-6}$). Наилучшее ускорение на 32 ядрах достигается MPI-версией ($\approx 9.4\times$) и гибридом ($\approx 9.3\times$), а OpenMP-версия, ограниченная одним узлом, даёт до $\approx 5.4\times$.